Word Vectors, Embeddings & Semantics in Vector Space

KAUST–Mawhiba IOAI Training

Week 3 · Day 13

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

May 4, 2026

Table of Contents

Recap: Day 12 – NLP Foundations

- NLP enables machines to process human language
- **Tokenization**: splitting text into meaningful units
- **Vocabulary**: the set of all unique tokens
- **N-grams**: capturing local word co-occurrence
- **Bag of Words**: representing documents as count vectors

Problem with BoW:

- High-dimensional and sparse
- "happy" and "joyful" are completely unrelated
- Common words like "the" dominate

Today's Roadmap

1. **TF-IDF**: smarter weighting for important words
2. **From Sparse to Dense**: one-hot $\rightarrow$ embeddings
3. **Similarity Measures**: Euclidean distance, cosine similarity
4. **Word Embeddings**: how they are learned (Word2Vec intuition)
5. **Using Embeddings**: in PyTorch and beyond

Goal: From counting words $\rightarrow$ understanding meaning in vector space.

TF-IDF: **Smarter Feature Extraction**

Our Running Example

Corpus of 3 movie reviews:

D1: "the movie was great and the acting was great"

D2: "the movie was terrible"

D3: "great acting and great story"

Vocabulary: {the, movie, was, great, and, acting, terrible, story}

We will use this same corpus for every step that follows.

Why Not Just Count Words?

Raw word counts for our corpus:

	the	movie	was	great	and	acting	terrible	story
D1	2	1	2	2	1	1	0	0
D2	1	1	1	0	0	0	1	0
D3	0	0	0	2	1	1	0	1

Problem: "the" and "was" get the highest counts in D1, but they tell us nothing about whether the review is positive or negative!

Idea: Weight words by how *unique* they are across documents.

- A word in *every* document $\rightarrow$ low weight
- A word in *few* documents $\rightarrow$ high weight

Step 1: Term Frequency (TF)

How often does word t appear in document d?

$$\text{TF}(t, d) = \frac{\text{count of } t \text{ in } d}{\text{total words in } d}$$

D1 has 9 words: "the movie was great and the acting was great"

Word	Count in D1	TF in D1
"the"	2	$2/9 = 0.22$
"great"	2	$2/9 = 0.22$
"was"	2	$2/9 = 0.22$
"movie"	1	$1/9 = 0.11$
"acting"	1	$1/9 = 0.11$

"the", "great", and "was" all have the same TF — but are they equally important?

Step 2: Inverse Document Frequency (IDF)

How rare is word t across all N documents?

$$\text{IDF}(t) = \log \frac{N}{|\{d : t \in d\}|}$$

Our corpus has $N = 3$ documents.

Word	In how many docs?	IDF
"the"	2 (D1, D2)	$\log(3/2) = 0.41$
"movie"	2 (D1, D2)	$\log(3/2) = 0.41$
"was"	2 (D1, D2)	$\log(3/2) = 0.41$
"great"	2 (D1, D3)	$\log(3/2) = 0.41$
"and"	2 (D1, D3)	$\log(3/2) = 0.41$
"acting"	2 (D1, D3)	$\log(3/2) = 0.41$
"terrible"	1 (D2 only)	$\log(3/1) = 1.10$
"story"	1 (D3 only)	$\log(3/1) = 1.10$

"terrible" and "story" are the rarest — they get the highest IDF!

Step 3: TF-IDF = TF × IDF

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

TF-IDF scores for document D1:

Word	TF (D1)	IDF	TF-IDF
"the"	0.22	0.41	0.090
"great"	0.22	0.41	0.090
"was"	0.22	0.41	0.090
"movie"	0.11	0.41	0.045
"acting"	0.11	0.41	0.045
"terrible"	0	1.10	0

TF-IDF for D2: "terrible" $= (1/4) \times 1.10 = \mathbf{0.275}$ — the highest score in D2!

The word that *distinguishes* D2 from the others gets the biggest weight.

The Full TF-IDF Matrix

Each document is now a weighted vector instead of raw counts:

	the	movie	was	great	and	acting	terrible	story
D1	.090	.045	.090	.090	.045	.045	0	0
D2	.102	.102	.102	0	0	0	**.275**	0
D3	0	0	0	.163	.082	.082	0	**.220**

Key insight: Compare with raw counts —

- Common words ("the", "was") are *down-weighted*
- Distinctive words ("terrible", "story") are *up-weighted*
- Each row is a document vector $\in \mathbb{R}^{|V|}$ ready for similarity, search, or classification

TF-IDF in Practice

Applications:

- **Search engines:** rank documents by relevance to a query
- **Text classification:** TF-IDF features $\rightarrow$ classifier
- **Document clustering:** group similar documents
- **Keyword extraction:** words with highest TF-IDF in a document

In Python (scikit-learn):

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
```

TF-IDF bridges the gap between raw counts (BoW) and learned embeddings (Word2Vec).

From Sparse to Dense: **Word Embeddings**

The Problem: Text is Symbolic

Models need numbers, but text is made of symbols.

Simplest approach: One-Hot Encoding

- Each word gets a unique vector with a single 1
- Vocabulary = {cat, dog, fish}

	cat	**dog**	**fish**
"cat"	1	0	0
"dog"	0	1	0
"fish"	0	0	1

Problems:

- **Huge:** vocabulary of 50K words → 50K-dimensional vectors
- **Sparse:** mostly zeros

Dense Embeddings: The Solution

Idea: Replace sparse one-hot vectors with short, dense vectors that are *learned from data*.

	dim 1	**dim 2**	**dim 3**	**...**
"cat"	0.2	-0.5	0.8	...
"dog"	0.3	-0.4	0.7	...
"fish"	0.1	-0.8	0.3	...
"airplane"	-0.9	0.6	-0.1	...

- Typically 100–300 dimensions (not 50,000!)
- **Similar words get similar vectors:** "cat" and "dog" are close
- **Dissimilar words are far apart:** "cat" and "airplane" are distant

The Famous Example: Word Arithmetic

Dense embeddings capture **relationships** between words:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

More examples:

- $\vec{\text{Paris}} - \vec{\text{France}} + \vec{\text{Italy}} \approx \vec{\text{Rome}}$
- $\vec{\text{walking}} - \vec{\text{walk}} + \vec{\text{swim}} \approx \vec{\text{swimming}}$

The embedding space encodes **meaning** — gender, geography, tense — as geometric directions!

How Are Embeddings Learned?

Core idea: "You shall know a word by the company it keeps."

Word2Vec (Google, 2013) — the most famous method:

Take a sentence: "The _____ sat on the mat"

Task: Predict the missing word from its neighbors.

- Context = {The, sat, on, the, mat} → Predict: "cat"

This is exactly like **masking**: hide a word, train a model to fill it in.

After training on millions of sentences, the model's internal weights become the word embeddings — words that appear in similar contexts get similar vectors.

Embedding Methods at a Glance

Method	Key Idea
Word2Vec	Predict a word from its neighbors (or vice versa)
GloVe	Learn from word co-occurrence counts across a whole corpus
fastText	Like Word2Vec but uses character pieces, so it handles rare/new words

In practice today:

- These "static" embeddings give one vector per word regardless of context
- Modern models like **BERT** and **GPT** produce **contextual** embeddings — the vector for "bank" changes depending on whether it means a river bank or a financial bank
- We will cover Transformers and BERT in upcoming lectures

Using Pre-trained Embeddings in PyTorch

You rarely train embeddings from scratch — use pre-trained ones!

```
import torch
import torch.nn as nn

# Create an embedding layer: 10K vocab, 300 dims
embed = nn.Embedding(num_embeddings=10000, embedding_dim=300)

# Look up the embedding for word index 42
word_idx = torch.tensor([42])
vector = embed(word_idx)  # shape: (1, 300)
```

Key point: An embedding layer is just a **lookup table** — given a word index, it returns the corresponding dense vector.

Tokenization Methods: **Beyond Words**

Three Ways to Tokenize

Sentence: “ChatGPT is amazing”

1. **Word-level:** [“ChatGPT”, “is”, “amazing”]
   - Simple, but what if “ChatGPT” is not in our vocabulary?
2. **Character-level:** [“C”, “h”, “a”, “t”, “G”, “P”, “T”, ...]
   - Handles any word, but each token carries almost no meaning
3. **Subword-level:** [“Chat”, “G”, “PT”, “is”, “amazing”]
   - Best of both worlds!

The Problem with Word-Level Tokenization

Imagine you trained a model on movie reviews.

Your vocabulary: {"good", "bad", "movie", "great", "terrible", ...}

Now someone writes: "This movie is **unbelievably** good"

- "unbelievably" was never in the training data!
- The model sees it as `<UNK>` — completely lost

But wait...

- "un" + "believ" + "ably" — each piece carries meaning!
- "un-" means negation, "believ" relates to "believe", "-ably" is an adverb suffix

What if we could split words into meaningful pieces automatically?

Byte Pair Encoding (BPE) — The Idea

BPE is the most popular subword tokenization method.

Used by GPT, LLaMA, and most modern language models.

How it works (simple version):

1. Start with individual characters as your vocabulary
2. Look at your text — find the pair of characters that appears **most often**
3. **Merge** that pair into a single new token
4. Repeat steps 2–3 until your vocabulary is big enough

That's it! BPE just keeps merging the most frequent pairs.

BPE Example — Step 0: Start with Characters

Training text: "low low low low lowest lowest wider wider wider"

Split every word into characters + end marker _:

"low" (4 times):	l o w _
"lowest" (2 times):	l o w e s t _
"wider" (3 times):	w i d e r _

Initial vocabulary: {l, o, w, _, e, s, t, i, d, r}

Now let's count which pair of neighbors appears most...

BPE Example — Step 1: First Merge

Count all adjacent pairs:

Pair	Count
(l, o)	$4 + 2 = 6$
(o, w)	$4 + 2 = 6$
(w, _)	4
(w, e)	2
(w, i)	3
(e, r)	3
...	...

Winner: (l, o) with 6 occurrences $\rightarrow$ merge into **"lo"**

Vocabulary: {l, o, w, _, e, s, t, i, d, r, **lo**}

BPE Example — Steps 2 & 3

After merging “lo”:

“low”: **lo** w _
“lowest”: **lo** w e s t _

Step 2: Most frequent pair is now (**lo**, w) with 6 → merge into **“low”**

“low”: **low** _
“lowest”: **low** e s t _

Step 3: Most frequent pair is (**low**, _) with 4 → merge into **“low_”**

“low”: **low_**
“lowest”: **low** e s t _

“low” is now a single token, but “lowest” still uses pieces!

BPE — Why It Works

After enough merges:

- Common words like “low” become single tokens
- Rare words like “lowest” get split into known pieces: “low” + “est”
- Brand new words work too: “lower” → “low” + “er”

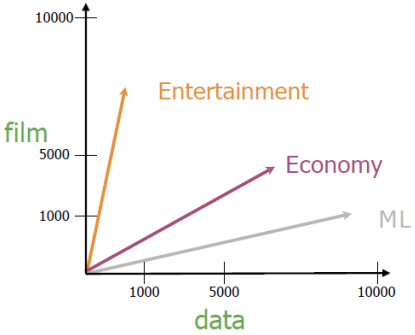
The magic:

- No more <UNK> — every word can be broken into known subwords
- Common words stay whole (efficient)
- Rare words share pieces with common words (generalize)
- “unbelievably” → [“un”, “believ”, “ably”] — each piece carries meaning

This is how ChatGPT, LLaMA, and all modern LLMs tokenize text!

Similarity Measures

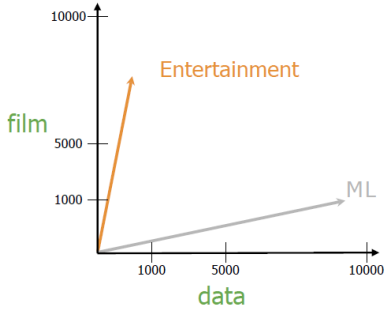
Vector Space

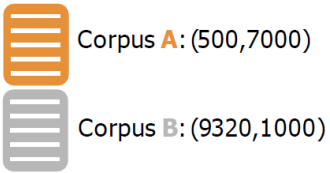


	Entertainment	Economy	ML
data	500	6620	9320
film	7000	4000	1000

Measures of "similarity:"
Angle
Distance

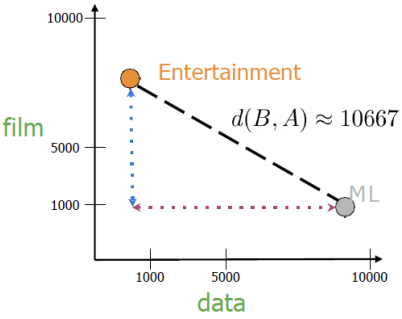
Euclidean Distance





- Measures the straight-line distance between two points in space.
- Formula: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$
- Sensitive to the scale (magnitude) of the vectors.

Euclidean Distance (cont.)



Corpus **A**: (500,7000)

Corpus **B**: (9320,1000)

$$d(B, A) = \sqrt{(B_1 - A_1)^2 + (B_2 - A_2)^2}$$

$$c^2 = a^2 + b^2$$

$$d(B, A) = \sqrt{(-8820)^2 + (6000)^2}$$

Euclidean Distance for n-dimensional vectors

	data	$\vec{w}$ boba	$\vec{v}$ ice-cream
AI	6	0	1
drinks	0	4	6
food	0	6	8

$$= \sqrt{(1-0)^2 + (6-4)^2 + (8-6)^2}$$

$$= \sqrt{1+4+4} = \sqrt{9} = 3$$

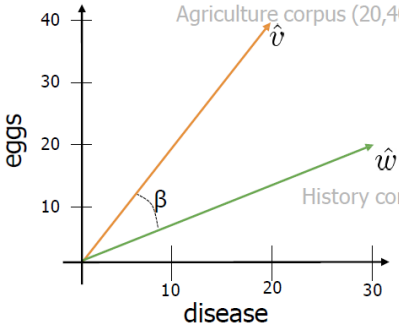
$$d(\vec{v}, \vec{w}) = \sqrt{\sum_{i=1}^{n} (v_i - w_i)^2} \longrightarrow \text{Norm of } (\vec{v} - \vec{w})$$

Euclidean Distance in Python

```
# Create numpy vectors v and w
v = np.array([1, 6, 8])
w = np.array([0, 4, 6])
# Calculate the Euclidean distance d
d = np.linalg.norm(v-w)
# Print the result
print("The Euclidean distance between v and w is: ", d)
```

```
The Euclidean distance between v and w is: 3
```

Cosine Similarity



$$\hat{v} \cdot \hat{w} = \|\hat{v}\|\|\hat{w}\| \cos(\beta)$$

$$\cos(\beta) = \frac{\hat{v} \cdot \hat{w}}{\|\hat{v}\|\|\hat{w}\|}$$

$$= \frac{(20 \times 30) + (40 \times 20)}{\sqrt{20^2 + 40^2} \times \sqrt{30^2 + 20^2}}$$

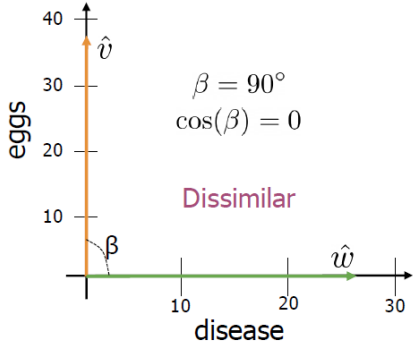
$$= 0.87$$

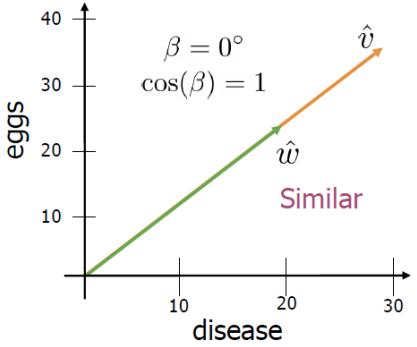
- Measures the cosine of the angle between two vectors.
- Formula: $\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|}$
- Ranges from -1 (opposite) to 1 (same direction).

Cosine Similarity (cont.)

- Less sensitive to magnitude, focuses on orientation.

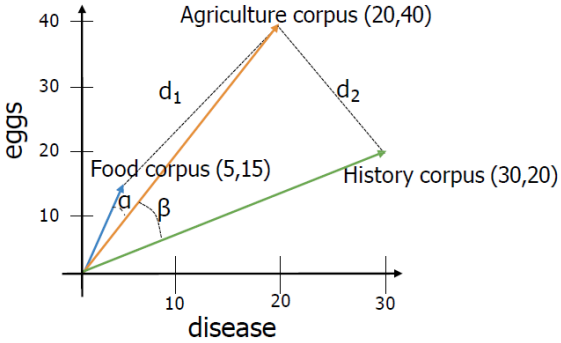
Cosine Similarity (cont.)





Euclidean distance vs Cosine similarity

- Cosine similarity when corpora are different sizes



Euclidean distance: $d_2 < d_1$

Angles comparison: $\beta > \alpha$

The cosine of the angle between the vectors

Manipulating word vectors

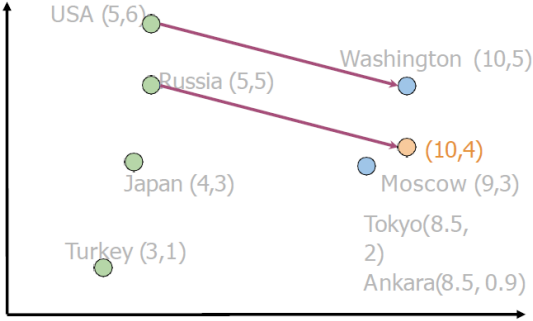
USA

Washington DC

Russia

?

Manipulating word vectors (cont.)



$$\text{Washington - USA} = \begin{bmatrix} 5 & -1 \end{bmatrix}$$

$$\text{Russia} + \begin{bmatrix} 5 & -1 \end{bmatrix} = \begin{bmatrix} 10 & 4 \end{bmatrix}$$

Moscow

[Mikolov et al, 2013, Distributed Representations of Words and Phrases and their Compositionality]

Visualization of word vectors

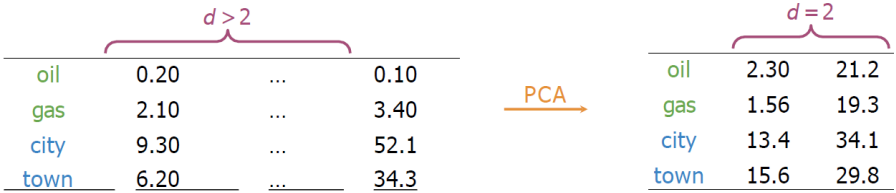
	$d > 2$		
oil	0.20	...	0.10
gas	2.10	...	3.40
city	9.30	...	52.1
town	6.20	...	34.3

How can you visualize if your representation captures these relationships?

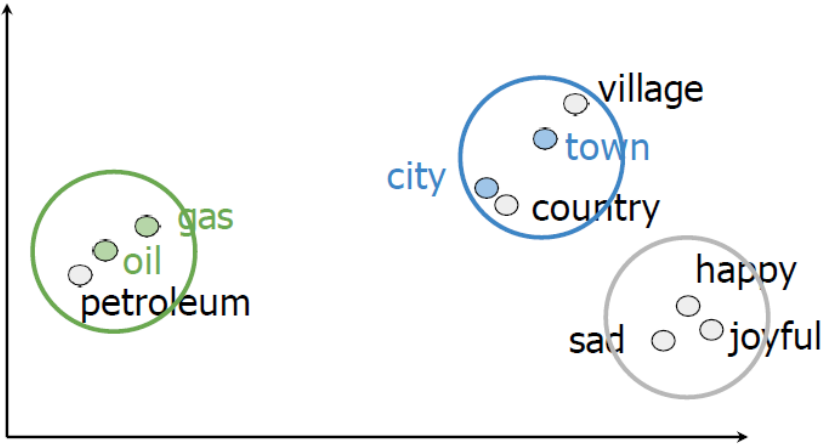
oil & gas

town & city

Visualization of word vectors (cont.)



Visualization of word vectors (cont.)



Key Takeaways

- **TF-IDF** weights words by importance: common words down, rare words up
- **One-hot encoding** is sparse and captures no meaning
- **Dense embeddings** represent words as short, learned vectors that encode semantic relationships
- **Similarity measures** (Euclidean distance, cosine similarity) let us compare vectors geometrically
- **Word2Vec** learns embeddings by predicting masked words from context — like a fill-in-the-blank game
- Modern models (**BERT**, **GPT**) produce contextual embeddings that change with sentence context

The Big Picture So Far

1. Raw text $\xrightarrow{\text{tokenize}}$ tokens
2. Tokens $\xrightarrow{\text{BoW / TF-IDF}}$ sparse vectors (good for classification)
3. Tokens $\xrightarrow{\text{embeddings}}$ dense vectors (capture meaning)
4. Dense vectors $\rightarrow$ feed into neural networks

Coming up:

- **Attention mechanism** — how models learn to focus on the right words
- **Transformers** — the architecture behind BERT, GPT, and modern AI

Credits

KAUST Academy

King Abdullah University of Science and Technology

Similarity measures content adapted from

KAUST Academy NLP course materials

KAUST–Mawhiba IOAI 2026 Training Program